Project - Low Level Design

On

Multimodal Insurance Design

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	S. JEZREEL	EN22IT301085
2	KRITAGYA TIWARI	EN22IT301048
3	SADHANA YADAV	EN24CA5030146
4	CHANDRESH BHATI	EN24CA50040
5	SHIVKUMAR SOLANKI	EN24CA5030157

Project Number:DO-40

Industry Mentor Name: Mr. Yash

University Mentor Name: Prof. Ajas Khan

Academic Year:2025-26

Group Name: 02D12

Table of Content

S. No.	Section / Title	Page No.
1	Introduction	3
1.1	Scope of the Document	3
1.2	Intended Audience	4
1.3	System Overview	4
1.4	System Architecture Diagram	5
2	System Design	6
2.1	Application Design	6
2.1.1	Class Diagram	6
2.1.2	Sequence Diagram	7
2.1.3	Activity Diagram	8
2.2	Process Flow	8
2.3	Information Flow	9
2.4	Components Design	9
2.5	Key Design Considerations	10
2.6	API Catalogue	10-13
3	References	14

Project Overview

Insurance Hub is a cloud-deployed insurance management platform built with Flask. It allows authenticated users to manage their insurance policies (Health, Term, Vehicle) and leverages Generative AI to explain insurance concepts and generate professional visual infographics — demonstrating the synergy between text-to-text and text-to-image AI models.

1. INTRODUCTION:

Insurance Hub is a full-stack web application designed to manage insurance policies and integrate **Multimodal Generative AI capabilities** for enhanced user experience. The system allows users to securely manage different types of insurance policies such as **Health Insurance, Term Insurance, and Vehicle Insurance**.

In addition to standard policy management features, the application includes an **AI Studio** that utilizes large language models and image generation models to explain insurance concepts and generate visual risk infographics.

The backend of the system is developed using **Python Flask**, while the AI functionality integrates **LLM APIs and text-to-image models**. The platform demonstrates how **Generative AI can be combined with traditional software systems** to provide intelligent and interactive features for users

1.1 SCOPE OF THE DOCUMENT:

This document defines the Low Level Design (LLD) of the Insurance Hub — Multimodal GenAI Insurance Management Application. It describes the internal structure and implementation details of the system, including modules, database schema, workflows, and integration with external AI services. The scope covers both the insurance policy management features (CRUD operations, authentication, and user data handling) and the AI Studio module, which utilizes text-to-text and text-to-image models to generate policy explanations and risk infographics. This document serves as a technical guide for understanding how the system processes user inputs, interacts with APIs, and produces outputs.

1.2 INTENDED AUDIENCE:

This document is intended for developers, testers, project evaluators, and system administrators who are involved in the design, development, deployment, or evaluation of the Insurance Hub — Multimodal GenAI Insurance Management Application. It helps developers understand the system architecture, module interactions, and API integrations, assists testers in validating system functionality and workflows, and enables evaluators to assess the correctness and effectiveness of the technical design. Additionally, it is useful for students and learners to understand the practical implementation of multimodal Generative AI systems, including the integration of LLM-based text generation and image generation models within a Flask-based web application.

1.3 SYSTEM OVERVIEW:

The **Multimodal Insurance Creator** is a Generative AI–powered system designed to transform **textual insurance concepts into both detailed explanations and visual representations**. The primary objective of the system is to demonstrate how **multimodal artificial intelligence models can work together** to create informative and visually engaging content related to insurance policies and risk concepts.

The application combines **text generation models (Large Language Models)** and **image generation models** through external APIs to produce two types of outputs from a single user prompt:

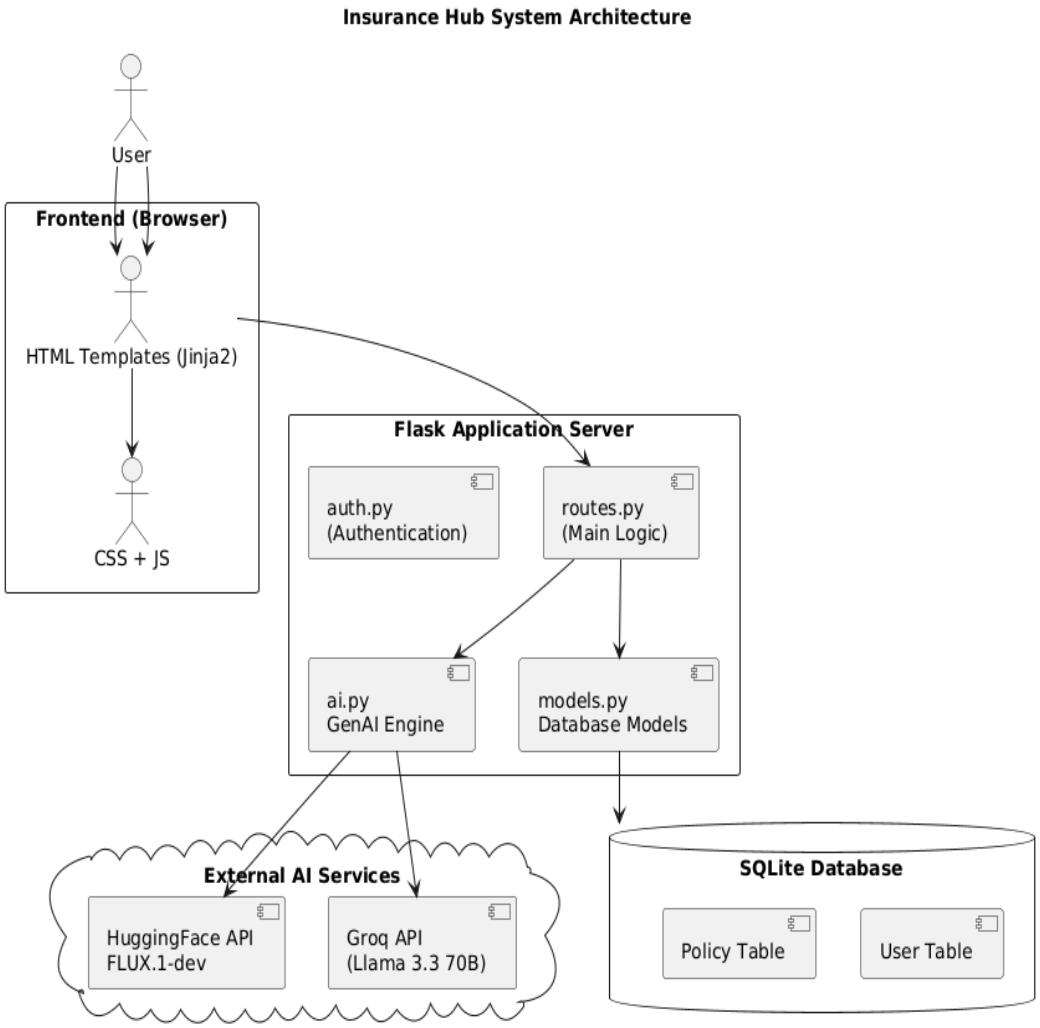
1. **Textual Explanation** — generated using a Large Language Model (LLM) that explains insurance concepts in a structured and easy-to-understand format.

2. **Visual Infographic** — generated using an AI image model that converts the same prompt into a professional infographic or conceptual visualization.

This dual-output approach illustrates the **synergy between text-to-text and text-to-image generative models**, allowing users to understand complex insurance topics through both **descriptive information and visual representation**.

1.4 SYSTEM ARCHITECTURE DIAGRAM:

The system architecture of the Insurance Hub application defines the overall structure of the system and explains how different components interact to deliver insurance management and multimodal AI capabilities. The architecture follows a layered web application design, where each layer performs a specific function such as user interaction, application processing, data storage, and AI service integration. This modular design improves maintainability, scalability, and security of the system



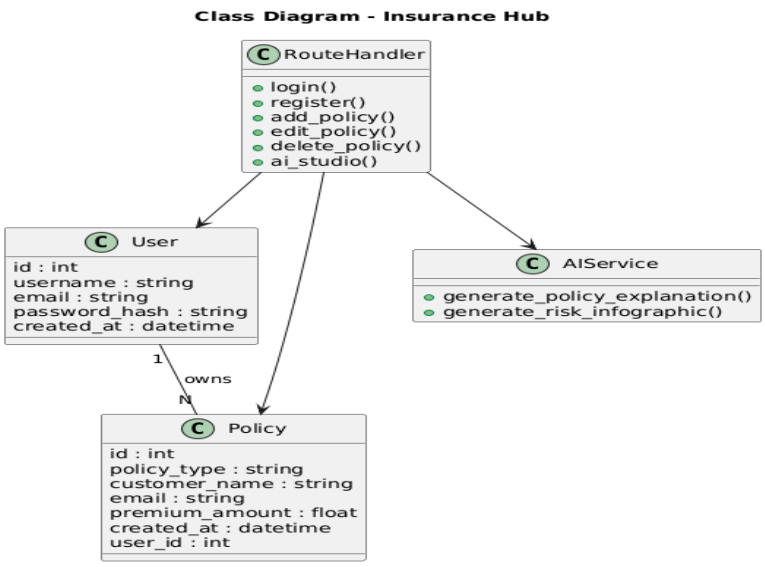
2. SYSTEM DESIGN

2.1 APPLICATION DESIGN

The *Insurance Hub* application follows a **modular and layered architecture** based on the Flask framework. The system is divided into three main layers: **presentation layer, application layer, and data layer**. The presentation layer consists of HTML templates rendered using Jinja2, providing an interactive user interface. The application layer handles business logic through Flask routes, authentication modules, and AI integration logic. The data layer uses SQLite with SQLAlchemy ORM to manage user and policy data. Additionally, the system integrates external AI services, including **LLM-based text generation and image generation APIs**, to provide multimodal outputs. This modular design ensures **scalability, maintainability, and clear separation of concerns**.

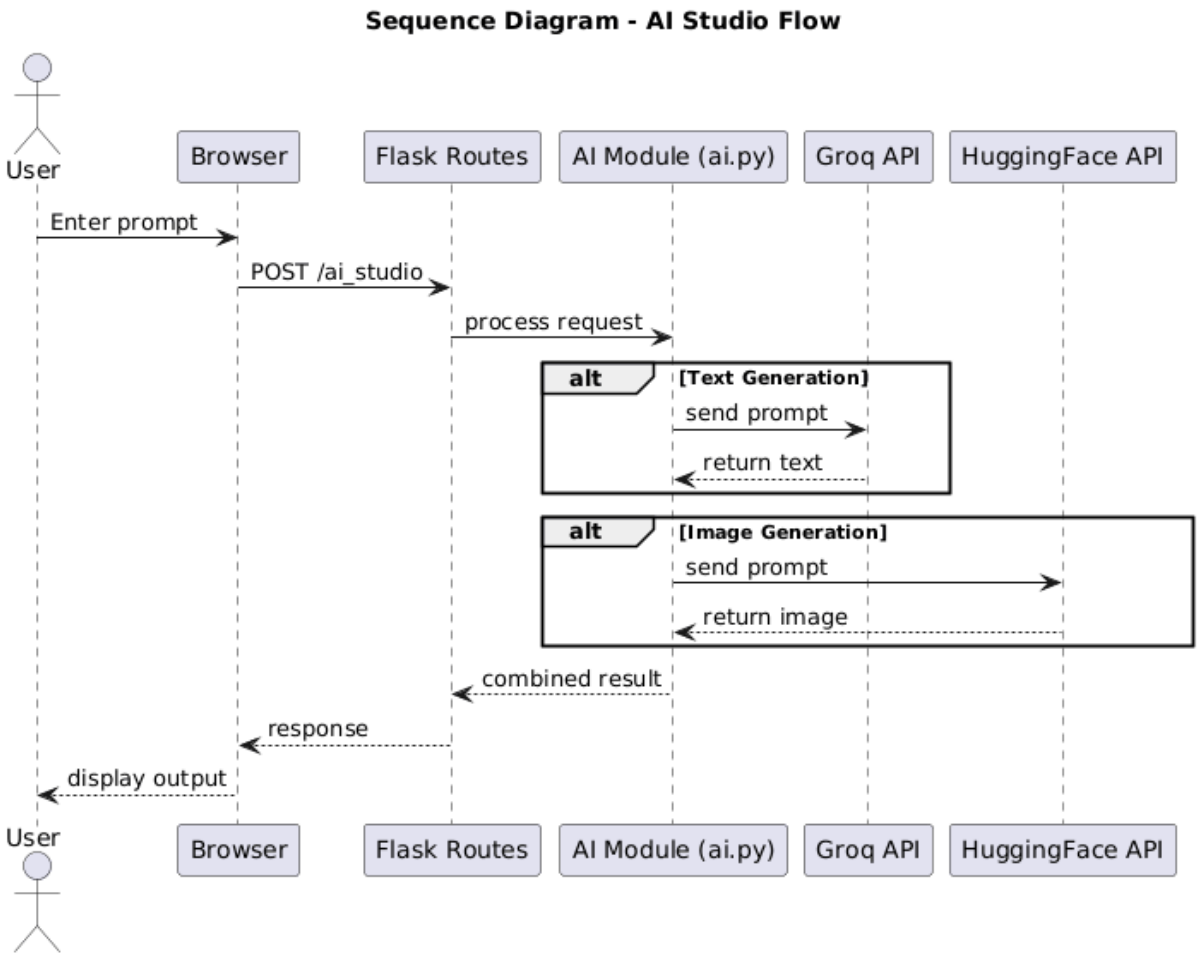
2.1.1 CLASS DIAGRAM

The class diagram represents the structural design of the Insurance Hub system. It includes core classes such as User and Policy for database representation, along with RouteHandler and AIService for application logic. The relationships show how users own policies and how the system interacts with AI services for generating text and image outputs.



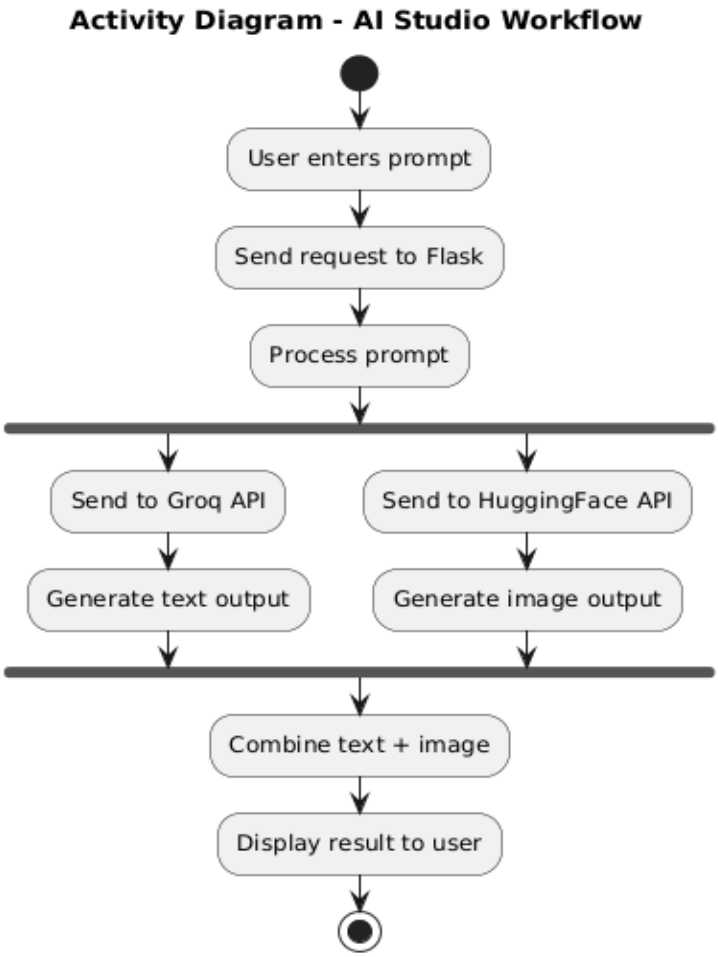
2.1.2 SEQUENCE DIAGRAM

The sequence diagram illustrates the interaction flow when a user requests AI-generated content. It shows how the request moves from the browser to Flask routes, then to the AI module, and further to external APIs like Groq and Hugging Face. The responses are processed and returned to the user interface.



2.1.3 ACTIVITY DIAGRAM

The activity diagram represents the workflow of the AI Studio feature. It begins with user input, followed by prompt processing, then splits into parallel activities for text and image generation. After both processes complete, the outputs are combined and displayed. It highlights the multimodal processing nature of the system.



2.2 PROCESS FLOW

The system process begins when a user interacts with the web interface. For policy management, the user performs operations such as adding, updating, or deleting policies, which are processed by Flask routes and stored in the database. For AI Studio features, the user provides a prompt related to an insurance concept. The system processes this input and sends it to the AI module. The prompt is then forwarded to two external services: the **LLM API for text generation** and the **image generation API for infographic creation**. The responses from both services are received, processed, and combined into a structured output, which is then displayed to the user. This flow ensures smooth interaction between user input, backend processing, and AI services.

2.3 Information Flow

The information flow in the system involves the movement of data between different components. User input data is first captured through the frontend interface and sent to the backend via HTTP requests. The backend validates and processes the data before either storing it in the database (for policy operations) or forwarding it to external AI APIs (for AI Studio features). The AI-generated responses are returned to the backend, formatted appropriately, and sent back to the frontend for display. Sensitive data such as user credentials are securely handled using password hashing and session management. This structured flow ensures **data integrity, security, and efficient communication between components**.

2.4 Component Level Design

2.4.1 Flask Application Components

backend consists of **five main modules**.

Module	File	Responsibility
App Factory	__init__.py	Initializes Flask, database, login manager
Authentication	auth.py	Register, login, logout
Main Logic	routes.py	Dashboard, CRUD operations, AI Studio
AI Engine	ai.py	LLM and image generation
Data Models	models.py	Database tables

2.5 Key Design Considerations

Several important design considerations were taken into account while developing the system. **Security** is ensured through password hashing, session management, and user data isolation. **Scalability** is achieved by designing the system in a modular way and supporting deployment through Docker and Kubernetes. **Performance** is optimized by using efficient API calls and handling AI responses asynchronously where possible. **Usability** is improved through a responsive interface and clear output formatting. **Reliability** is maintained by handling API failures and providing fallback mechanisms. These considerations ensure that the system is robust, secure, and user-friendly.

2.6 API Catalogue

The Insurance Hub application integrates both **external AI APIs** and **internal application APIs** to deliver its core functionalities. The external APIs are responsible for **multimodal AI processing**, while the internal APIs (Flask routes) handle **authentication, policy management, and AI request handling**.

2.6.1 LLM API (Text-to-Text Generation)

The system uses the Groq API to generate structured insurance explanations through a Large Language Model.

- **Service:** Groq API
- **Model:** Llama 3.3 70B Versatile
- **Purpose:** Generate detailed and structured insurance policy explanations
- **Module Used:** ai.py → generate_policy_explanation()

INPUT

User prompt (e.g., *"Explain health insurance benefits"*)

PROCESSING

- Prompt is enhanced with a system context (Indian insurance domain)
- Sent to LLM via API request

OUTPUT

- Structured text response:
  - Overview
  - Key Benefits
  - Important Information
  - Premium Range

2.6.2 Image Generation API (Text-to-Image)

The system uses the Hugging Face Inference API for generating insurance infographics.

- **Service:** Hugging Face InferenceClient
- **Model:** FLUX.1-dev
- **Purpose:** Generate professional insurance risk infographics
- **Module Used:** ai.py → generate_risk_infographic()

INPUT

- Text prompt describing visual concept
  (e.g., *"Infographic showing health insurance risk coverage"*)

PROCESSING

- Prompt sent to image model
- Model generates image (PIL format)
- Image converted to **Base64 format**

OUTPUT

- Embedded image displayed directly in UI (no external URL dependency)

2.6.3 Internal Application APIs (Flask Routes)

The application defines multiple **internal APIs using Flask routes**, which handle user interaction, data processing, and AI integration.

Authentication APIs

Endpoint	Method	Description
/login	POST	Authenticates user and starts session
/register	POST	Registers a new user account
/logout	GET	Logs out the current user

Policy Management APIs

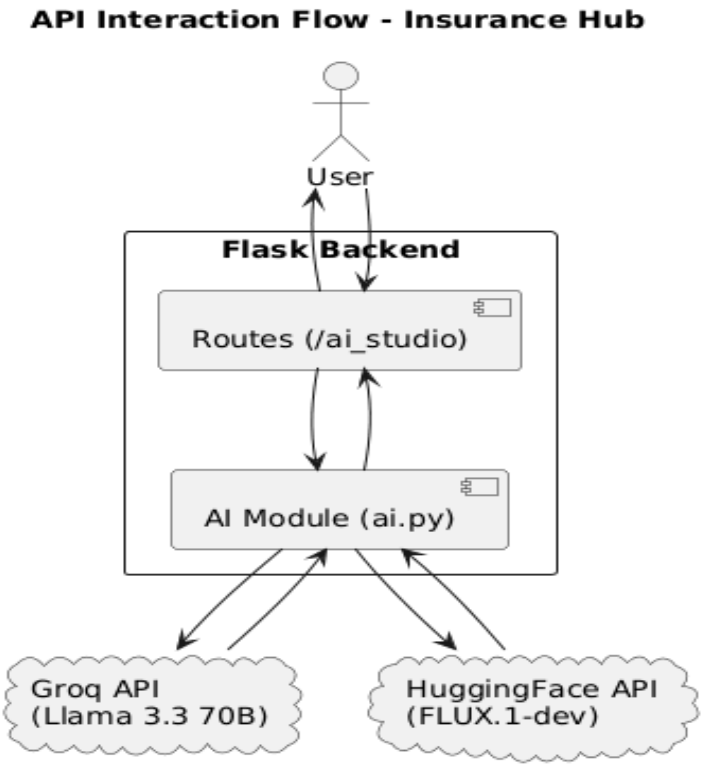
Endpoint	Method	Description
/add_policy	POST	Adds a new insurance policy
/edit_policy	POST	Updates existing policy details

/delete_policy	POST	Deletes a policy
/	GET	Displays user dashboard with policies

AI Studio API

Endpoint	Method	Description
/ai_studio	POST	Handles AI requests (text + image generation)

2.6.4 API Interaction Flow



REFERENCES

1. Flask Framework Documentation

The Flask documentation was used for building the backend web application, routing system, template rendering, and server configuration.

https://flask.palletsprojects.com/

3. Flask-Login Documentation

Flask-Login was used for implementing **user authentication and session management**.
https://flask-login.readthedocs.io/

4. Groq API Documentation

The Groq API was used for **LLM-based text generation** in the Policy Explainer module.
https://console.groq.com/docs

5. Hugging Face Inference API

The Hugging Face inference client was used for **image generation using FLUX.1-dev**.
Hugging Face
https://huggingface.co/docs

6. SQLAlchemy Documentation

SQLAlchemy was used as the **Object Relational Mapper (ORM)** for interacting with the database.

https://docs.sqlalchemy.org/

7. Docker Documentation

Docker was used for **containerizing the application** before deployment.

https://docs.docker.com/

8. Kubernetes Documentation

Kubernetes was used for **container orchestration and deployment**.

https://kubernetes.io/docs/

9. Python Official Documentation

Python was the primary programming language used for backend development.

https://docs.python.org/3/